# 🛑 ¡ALTO! ESTA APLICACIÓN ESTÁ PROTEGIDA POR SPRING SECURITY

Actuación educativa para 3 personajes · Duración aproximada: 5–6 minutos

Escenario: La entrada de una aplicación web llamada JavaBank. Spring Security controla quién puede ingresar y qué acciones puede realizar.

## Personajes

• Spring Security: el guardián de la aplicación; estricto y algo exagerado.

• Usuario: quiere ingresar a su cuenta y consultar su saldo.

• Hacker/Bot: intenta entrar sin autorización.

## ESCENA 1. La entrada a JavaBank

(Spring Security está parado frente a una puerta imaginaria con un cartel que dice: «Aplicación Java protegida».)

Spring Security: ¡Alto! ¡Nadie entra sin autorización! Bienvenidos a JavaBank, una aplicación desarrollada en Java con Spring Boot. Yo soy Spring Security, el encargado de proteger esta aplicación.

Usuario: ¡Buenos días! Vengo a ingresar a mi cuenta para consultar mi saldo.

Spring Security: Muy bien, ciudadano. Antes de entrar necesito comprobar quién eres y qué permisos tienes.

Usuario: ¿Y cómo haces eso?

Spring Security: ¡Fácil! Primero está el SecurityFilterChain, mi equipo de vigilancia. Se encarga de interceptar las solicitudes HTTP antes de que lleguen a los controladores de la aplicación.

Usuario: ¿O sea que revisas mi solicitud antes de dejarme entrar?

Spring Security: ¡Exactamente! Aquí no entra ni una petición sin pasar por mi filtro.

## ESCENA 2. El usuario presenta sus credenciales

Usuario: Bueno, aquí tengo mi nombre de usuario y contraseña.

(Entrega un papel imaginario.)

Spring Security: ¡Perfecto! Ahora entra en acción el AuthenticationManager, que coordina el proceso de autenticación.

Usuario: ¿Y quién comprueba mis datos?

Spring Security: Para eso tenemos al UserDetailsService, que busca la información del usuario, y al PasswordEncoder, que verifica tu contraseña comparándola de forma segura con su versión almacenada mediante un algoritmo de hash.

Usuario: ¡Qué complicado! ¿No puedes simplemente preguntarme mi contraseña y ya?

Spring Security: ¡Claro que no! ¡Esto es seguridad, no una reunión de amigos!

(Spring Security revisa sus documentos.)

Spring Security: ¡Excelente! Tus credenciales son correctas. Has sido autenticado.

Usuario: ¿Eso significa que ya puedo hacer todo?

Spring Security: ¡No tan rápido! Saber quién eres no significa que tengas permiso para hacer cualquier cosa.

## ESCENA 3. Aparece el hacker

(Entra el hacker caminando sospechosamente, con lentes oscuros y una laptop imaginaria.)

Hacker: ¡Muajajaja! ¡Vengo a entrar a JavaBank! Mi usuario es: administrador, y mi contraseña es: 123456.

Spring Security: ¿123456? ¡Hasta mi abuela utiliza contraseñas más seguras!

Hacker: ¡Pero soy un bot! ¡Puedo probar miles de contraseñas!

Spring Security: ¡Pues te tengo una sorpresa! Mis filtros también pueden proteger las solicitudes y trabajar con mecanismos de seguridad como la protección CSRF, que ayuda a evitar solicitudes falsas realizadas en nombre de usuarios autenticados.

Hacker: ¡Bueno, bueno! Entonces entraré directamente a la dirección de administración.

(El hacker señala una dirección imaginaria: javabank.com/admin.)

Spring Security: ¡DETENTE! Aquí aparece otro de mis componentes: la autorización.

Hacker: ¿Autorización? ¿No es lo mismo que autenticación?

Spring Security: ¡Por supuesto que no! La autenticación comprueba quién eres; la autorización determina qué puedes hacer.

Hacker: ¡Pero yo quiero ser administrador!

Spring Security: ¡Y yo quiero vacaciones! Pero ninguno de los dos tendrá lo que quiere.

(El hacker intenta entrar y Spring Security le bloquea el paso.)

Spring Security: Para acceder a determinadas funciones necesitas roles o autoridades. Por ejemplo, un usuario con el rol USER puede consultar su cuenta, mientras que uno con el rol ADMIN puede gestionar usuarios.

Hacker: ¡Entonces no tengo permiso!

Spring Security: ¡Correcto! Acceso denegado: error 403.

## ESCENA 4. El usuario finalmente ingresa

Usuario: ¡Entonces yo sí puedo entrar porque mis datos fueron comprobados!

Spring Security: ¡Correcto! Además, puedo mantener tu autenticación mediante una sesión o utilizar tokens, dependiendo de cómo esté configurada la aplicación.

Usuario: ¡Excelente! Entonces quiero consultar mi saldo.

Spring Security: Como tienes el rol USER, tienes autorización para acceder a esa función.

Usuario: ¡Genial! ¿Y si intento eliminar a todos los usuarios?

Spring Security: ¡Ni lo sueñes! Esa función está reservada para el administrador.

Hacker: ¡Esto es injusto! ¡Yo también quiero entrar!

Spring Security: ¡Ya te dije que no! ¡Y deja de intentar poner 123456 como contraseña!

## ESCENA 5. El mensaje final

(Los tres personajes se colocan frente al público.)

Usuario: Ahora entiendo que Spring Security protege una aplicación Java Web verificando las credenciales de los usuarios.

Hacker: Y también aprendí que no basta con conocer una dirección web o adivinar una contraseña para acceder a cualquier función.

Spring Security: ¡Exactamente! Recuerden:

- SecurityFilterChain: intercepta y organiza los filtros de seguridad.
- AuthenticationManager: coordina el proceso de autenticación.
- UserDetailsService: obtiene los datos del usuario.
- PasswordEncoder: verifica contraseñas de forma segura.
- Authentication: representa al usuario autenticado y sus autoridades.
- Authorization: controla los permisos y roles.
- CSRF: ayuda a proteger contra solicitudes falsificadas.

Usuario: ¡Entonces Spring Security no solamente controla quién entra, sino también qué puede hacer dentro de la aplicación!

Spring Security: ¡Así es! Porque una aplicación segura no consiste solamente en cerrar la puerta, sino en asegurarse de que cada persona tenga acceso únicamente a lo que le corresponde.

Hacker: ¡Bueno, me voy a buscar una aplicación sin seguridad!

Spring Security: ¡Y nosotros vamos a buscarte una contraseña más difícil! ¡FIN!

## Guía rápida: componentes y su función

| Componente | Función |
|---|---|
| SecurityFilterChain | Define los filtros que procesan las solicitudes HTTP. |
| AuthenticationManager | Coordina el proceso de autenticación. |
| UserDetailsService | Recupera los datos del usuario desde una fuente de información. |
| PasswordEncoder | Comprueba contraseñas utilizando algoritmos de hash. |
| Authentication | Representa al usuario autenticado y sus autoridades. |
| Authorization | Determina si el usuario tiene permiso para acceder a un recurso. |
| CSRF Protection | Ayuda a prevenir solicitudes falsificadas. |

## Consejos para la representación

- Spring Security: puede utilizar lentes, una gorra de policía o una credencial de seguridad.
- Usuario: puede llevar un papel con su nombre de usuario y contraseña.
- Hacker: puede utilizar lentes oscuros, una capucha y una laptop de cartón.
- Escenario: una silla o mesa puede representar la entrada de JavaBank.

• Efecto cómico: cuando el hacker intente entrar, Spring Security puede gritar «¡ACCESO DENEGADO!» y mostrar un cartel rojo.

Consejo para la exposición: pronuncien claramente los nombres técnicos de los componentes. Así la actuación será entretenida y también demostrará cómo se utiliza Spring Security en una aplicación Java Web.